

Taller 1: Búsqueda con y sin información

Camilo Puerto Peña - 202211362

Daniela Munar González - 202311392

Nicolas Hurtado - 202414730

Andres Tello - 20222807

Tabla de contenido:

Punto 1: Diagnóstico inicial mediante DFS.....	1
Punto 2: Ruta más corta mediante BFS.....	3
Punto 3 y 4: Navegación de menor costo mediante UCS con módulo obligatorio y costo dependiente del estado.....	4
Punto 5: A* y diseño de heurísticas.....	6

Punto 1: Diagnóstico inicial mediante DFS

a) depthFirstSearch en algorithms/search.py:

En este punto el problema consiste en llevar al robot desde su posición inicial hasta la terminal de diagnóstico. Para resolverlo nosotros pensamos en crear `start = problem.getStartState()`, que guarda el punto donde comienza el robot. También creamos `stack`, que guarda los caminos que el robot puede seguir, y `visited`, que guarda las posiciones por las que ya pasó para evitar que vuelva a explorarlas y se quede en ciclos. Así implementamos una búsqueda en profundidad, porque el robot sigue avanzando por un camino hasta donde pueda antes de explorar otros caminos posibles.

Mientras existan caminos por explorar, sacamos un estado de `stack` y revisamos si corresponde a la terminal usando `problem.isGoalState(state)`. Si llegamos a la terminal retornamos `path`, que contiene todos los movimientos realizados. Si todavía no llegamos, usamos `problem.getSuccessors(state)` para obtener los siguientes movimientos posibles y los agregamos a `stack`. De esta forma el robot continúa explorando hasta encontrar una ruta válida hacia la terminal.

b) Clase DiagnosticProblem y prueba de implementación:

```
Ejecutando:
C:\Users\dany\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFou
z5n2kfra8p0\python.exe C:\Users\dany\Documents\U.ANDES\IA\EstacionEsp
osticProblem -f depthFirstSearch -l tinyDiagnostic -x 0.25 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 107 in 0.0 seconds
Search nodes expanded: 83
¡Misión completada! Costo: 107

Ejecucion finalizada.
Presiona Enter para volver al menu...
```

Al ejecutar el código pudimos ver en la interfaz gráfica cómo el robot recorrió el mapa desde su posición inicial hasta llegar a la terminal de diagnóstico D. La misión se completó correctamente y el camino encontrado tuvo un costo total de 107. Esto muestra que DFS logró encontrar una ruta válida hasta el objetivo, que era lo que se buscaba en este punto.

En los resultados de la consola podemos ver que el algoritmo expandió 83 nodos y encontró el camino en 0.0 segundos. El costo de 107 se debe al camino que DFS decidió explorar, ya que en este punto no estamos buscando el camino de menor costo, sino simplemente encontrar una ruta que permita llegar a la terminal. Por esto, el robot puede pasar por diferentes tipos de corredores y terminar con un costo alto, pero aun así completar correctamente la misión.

c) Reflexión sobre el trabajo realizado:

- **Complejidad temporal y espacial:** En DFS usamos visited para evitar explorar varias veces una misma posición. La complejidad temporal es $O(V + E)$, donde V son los estados visitados y E los movimientos posibles entre ellos. En espacio puede llegar a $O(V)$ por los estados guardados en visited y stack.
- **Complejidad:** En los mapas del taller DFS es completo porque existe una cantidad limitada de posiciones y visited evita que el robot entre en ciclos. Por esto, si existe una ruta hasta la terminal D, DFS puede encontrarla.
- **Optimalidad:** DFS no garantiza una solución óptima, porque simplemente busca cualquier camino válido. Esto se pudo ver en nuestra ejecución, donde se encontró una solución de costo 107 expandiendo 83 nodos, sin comprobar si existía una ruta de menor costo.

Reflexión sobre el uso de inteligencia artificial

Prompt utilizado: “Revisa nuestra implementación de búsqueda y propón una versión mejorada sin cambiar la idea principal del algoritmo. Intenta mejorar el uso de memoria y el manejo de los estados visitados, manteniendo el código sencillo.”

La IA mantuvo la idea principal de nuestra solución, pero sugirió mejorar el manejo de visited y de los estados que se agregan para explorar. Esto nos ayudó a entender cómo evitar guardar estados repetidos innecesariamente. Los cambios fueron fáciles de entender porque partían de la misma lógica que ya habíamos implementado.

Punto 2: Ruta más corta mediante BFS

a) breadthFirstSearch en algorithms/search.py:

En este punto el problema consiste en llevar al robot desde su posición inicial hasta la terminal de diagnóstico, pero ahora buscando la ruta con la menor cantidad de movimientos. Para resolverlo nosotros creamos `start = problem.getStartState()` para guardar la posición

inicial, queue para guardar los caminos que faltan por explorar y visited para registrar las posiciones que ya fueron revisadas y así evitar repetirlas.

Mientras existan elementos en queue, sacamos el primero usando pop(0) y comprobamos si ese estado es la meta. Si no lo es, obtenemos sus sucesores con problem.getSuccessors(state) y agregamos los que todavía no han sido visitados al final de queue. Así implementamos una búsqueda por amplitud, porque el robot revisa primero las posiciones que están más cerca y luego continúa con las siguientes, permitiendo encontrar la solución con menor cantidad de movimientos, que es justamente lo pedido en el punto 2.

b) Clase DiagnosticProblem y prueba de implementación:

```
Ejecutando:
C:\Users\dany\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe C:\Users\dany\Documents\U.ANDES\IA\EstacionEspacial\main.py -p DiagnosticProblem -f breadthFirstSearch -l tinyDiagnostic -x 0.25 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 147
¡Misión completada! Costo: 24
```

Al ejecutar BFS con DiagnosticProblem, el robot logró llegar correctamente desde su posición inicial hasta la terminal D. La ruta encontrada tuvo un costo total de 24 y la ejecución tomó 0.0 segundos. En este caso el algoritmo expandió 147 nodos, ya que BFS va revisando las posiciones por niveles hasta encontrar la ruta con la menor cantidad de movimientos.

Comparándolo con DFS, podemos ver que BFS expandió más nodos, 147 frente a 83, pero encontró una ruta con un costo mucho menor, 24 frente a 107. Esto ocurre porque BFS busca primero las rutas con menos movimientos, mientras que DFS sigue una rama hasta donde pueda y retorna la primera solución válida que encuentre.

c) Reflexión sobre el trabajo realizado:

- **Complejidad temporal y espacial:** BFS guarda los estados visitados y los que todavía están pendientes en queue. Con nuestra implementación, la búsqueda debe revisar los estados y sus posibles movimientos, por lo que usamos $O(V + E)$ en tiempo y $O(V)$ en espacio.
- **Complejidad:** BFS es completo en los mapas del taller porque hay una cantidad limitada de posiciones y usamos visited para evitar ciclos. Si existe un camino hacia D, el algoritmo puede encontrarlo.
- **Optimalidad:** BFS encuentra una solución óptima cuando lo que queremos minimizar es la cantidad de movimientos. En este punto esto funciona porque precisamente se pide encontrar la ruta con menos pasos.

Reflexión sobre el uso de inteligencia artificial

Prompt utilizado: “Revisa nuestra implementación de breadthFirstSearch y busca si se puede mejorar o hacer más clara sin cambiar el funcionamiento de BFS ni la forma en que encuentra el camino más corto.”

La IA mantuvo prácticamente la misma lógica de nuestro código. El principal cambio fue hacer algunos nombres más claros, como usar `nextState` para representar el siguiente estado. Esto nos ayudó a confirmar que la forma en que manejábamos `queue` y `visited` ya seguía correctamente la idea de BFS.

Punto 3 y 4: Navegación de menor costo mediante UCS con módulo obligatorio y costo dependiente del estado

a) `uniformCostSearch` en `algorithms/search.py`:

En este punto el problema consiste en encontrar una ruta de menor costo usando UCS. Para resolverlo usamos `start = problem.getStartState()` para obtener la posición inicial, `priorityQueue` para guardar los estados pendientes según su costo, `bestCost` para guardar el menor costo encontrado y `cameFrom` para poder reconstruir el camino. UCS va sacando primero el estado que tenga el menor costo acumulado y, para cada sucesor, calcula `newCost = currentCost + stepCost`. Si encuentra una forma más barata de llegar a un estado, actualiza su costo y lo vuelve a guardar en la cola.

Para el punto 4 reutilizamos este mismo UCS, pero ahora completamos `ModuleRepairProblem`. En `isGoalState` verificamos que el robot no solo llegue al centro de control C, sino que también haya recogido antes el módulo M. En `_getStepCost` manejamos el cambio de costo de los movimientos, ya que antes de recoger el módulo se usa el costo normal y, después de recogerlo, los siguientes movimientos cuestan el doble. Finalmente, en `getSuccessors` generamos los posibles movimientos del robot y retornamos cada uno con el formato `(nextState, direction, stepCost)`, teniendo en cuenta tanto la nueva posición como si el robot ya tiene el módulo.

b) Clase `DiagnosticProblem` y prueba de implementación:

```
Ejecutando:
C:\Users\dany\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe C:\Users\dany\Documents\U.
NDES\IA\EstacionEspacial\main.py -p DiagnosticProblem -f uniformCostSearch -l tinyDiagnostic -x 0.25 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 132
¡Misión completada! Costo: 24
```

En la primera prueba usamos `DiagnosticProblem` con UCS para llegar a la terminal D. El algoritmo encontró una solución con costo 24, expandió 132 nodos y tomó 0.0 segundos. En este caso UCS logró encontrar una ruta de menor costo teniendo en cuenta los diferentes costos de los corredores.

```

C:\Users\danyam\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe C:\Users\danyam\Documents\IA\EstacionEspacial\main.py -p ModuleRepairProblem -f uniformCostSearch -l tinyModuleRun -x 0.25 -z 1.0

RobotPositions: [(1, 3)]
MissionSymbols: {'D': [], 'C': [(13, 4)], 'M': [(6, 8)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 48 in 0.0 seconds
Search nodes expanded: 139
Misión completada! Costo: 48
Ejecucion finalizada.

```

En la segunda prueba usamos ModuleRepairProblem, donde el robot debía recoger primero M y después llevarlo hasta C. La misión también se completó correctamente con un costo total de 48, 139 nodos expandidos y un tiempo de 0.0 segundos. El costo es mayor porque en esta misión no basta con llegar directamente a C: primero se debe visitar M y, después de recogerlo, los siguientes movimientos tienen el doble de costo.

c) Reflexión sobre el trabajo realizado:

- **Complejidad temporal y espacial:** Para UCS podemos representar la complejidad temporal como $O((V + E) \log V)$, donde V son los estados y E los movimientos posibles entre ellos. La complejidad espacial es $O(V)$, ya que se deben guardar los estados pendientes, sus costos y la información necesaria para reconstruir el camino.
- **Complejitud:** En los problemas del taller UCS puede encontrar una solución si existe una ruta válida, ya que los movimientos tienen costos positivos y se controla el mejor costo encontrado para cada estado
- **Optimalidad:** UCS encuentra la solución de menor costo porque siempre da prioridad al camino que tenga el menor costo acumulado. Esto es especialmente importante en los puntos 3 y 4 porque los movimientos pueden tener costos diferentes.

Reflexión sobre el uso de inteligencia artificial

Prompt utilizado: “Revisa nuestra implementación de uniformCostSearch y mejora la forma en que se manejan los costos y se reconstruye el camino, pero sin cambiar la lógica principal de UCS. Mantén el código sencillo y fácil de entender.”

La IA mantuvo la misma idea principal de nuestra solución, pero hizo más claro el manejo de los costos y de los nombres de las variables, por ejemplo usando tentativeCost y costs. También separó mejor la reconstrucción del camino con reconstructPath. Esto nos ayudó a entender mejor cómo UCS guarda el mejor costo conocido para cada estado y cómo recupera la ruta al llegar a la meta.

d) Explique por qué la posición del robot no es suficiente para representar el problema del punto 4 y por qué hasModule debe formar parte del estado:

En el punto 4 no es suficiente guardar solamente la posición del robot, porque estar en una misma posición puede representar dos situaciones diferentes: que el robot todavía no tenga el módulo M o que ya lo haya recogido. Esto cambia el problema porque después de recoger M los siguientes movimientos cuestan el doble y, además, llegar a C solo completa la misión si el robot ya tiene el módulo. Por eso usamos el estado (position, hasModule), donde position indica dónde está el robot y hasModule permite saber si ya recogió el módulo.

Punto 5: A* y diseño de heurísticas

a) aStarSearch en algorithms/search.py:

El problema consiste en guiar el robot hacia la meta minimizando el costo total usando la función $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado desde el inicio hasta el nodo n , y $h(n)$ es la estimación que hace la heurística hasta el objetivo. Para resolverlo usamos una cola de prioridad (pq) gestionando con (heapq) y un conjunto (explored) para controlar los nodos que se expandieron. Insertamos el estado inicial en la cola, con una tupla (total_f, accumulated_g, current_state, current_path) donde la prioridad inicial es $f(\text{start}) = 0 + \text{heuristic}(\text{start}, \text{problem})$, el costo acumulado (g) es 0 y el camino recorrido en este inicio es una lista vacía.

Tras cada iteración se extrae el elemento con menor $f(n)$ mediante `heapq.heappop`. Si el estado ya está en (explored), lo descartamos para no ser redundantes. Si el estado actual cumple la prueba de meta (`problem.isGoalState(current_state)`), retornamos inmediatamente la lista de acciones. En caso contrario, marcamos el estado como explorado añadiéndolo a (explored) y generamos sus sucesores usando (`problem.getSucesors(current_state)`), para cada sucesor no explorado, calculamos su costo real acumulado y su costo total proyectado ($f'(n)$) insertando la tupla actualizada en pq.

Reflexión sobre el trabajo realizado:

- **Complejidad temporal y espacial:** Complejidad temporal y espacial: La complejidad temporal en el peor caso es $O(b^d)$, reduciéndose significativamente con una heurística informada. En espacio requiere $O(b^d)$ para almacenar los estados en la cola de prioridad y en el conjunto.
- **Complejitud:** Es completo en espacios de estados finitos con costos de paso positivos.
- **Optimalidad:** Garantiza la solución de costo mínimo siempre que la heurística sea admisible y consistente.

Reflexión sobre el uso de inteligencia artificial

Prompt utilizado (aStarSearch): “Revisa esta implementación de A* para un problema de búsqueda en cuadrícula. Sugiere mejoras en estructura, manejo de la cola de prioridad.. Mantén la lógica y simplicidad del código”

La IA mencionó una buena implementación del algoritmo. Ofreció 2 mejoras: En primer lugar un bug que se puede generar en la cola de prioridad en caso de empate, si el estado es algo difícil de comparar, python puede lanzar error. En segundo lugar la complejidad de (`current_path + [move]`) que en caminos largos se puede volver $O(n^2)$.

b) manhattanHeuristic en algorithms/heuristics.py :

En este punto calculamos la distancia Manhattan hacia el objetivo inmediato según $|x1 - x2| + |y1 - y2|$.

En la implementación extraemos la tupla de estado (pos, hasKit, pendingSyst) y evaluamos la fase en la que está:

1. Si el robot no tiene el kit (not hasKit): El objetivo inmediatamente es (problem.kitPosition), retornando la suma de diferencias absolutas X y Y.
2. Si ya tiene el kit pero (len(pendingSystems) > 0), calculamos la distancia Manhattan a cada coordenada en (pendingSystems) y seleccionamos el valor mínimo usando (min(...)).
3. Si todos los sistemas han sido reparados, el objetivo final es el centro de control (problem.controlPosition), calculando la distancia directa hacia esa celda.

c) euclideanHeuristic en algorithms/heuristics.py :

En este punto calculos la distancia euclidiana en linea recta hacia el objetivo de la fase

$$(\sqrt{(x1 - x2)^2 + (y1 - y2)^2}).$$

En la implementación se aplica la misma lógica del anterior por etapas:

1. Sin kit (not hasKit): Calculamos $\sqrt{(x1 - xn)^2 + (y1 - yn)^2}$ hacia (problem.kitPosition).
2. Con kit y sistemas pendientes: calculamos la distancia euclidiana hacia cada sistema en (pendingSystems) y tomamos la mínima.
3. Si todos los sistemas estan reparados: calculamos la distancia euclidiana directa hacia problem.controlPosition.

Reflexión sobre el trabajo realizado (Manhattan y euclidean):

- **Complejidad temporal y espacial:** Complejidad temporal y espacial: La complejidad temporal en el peor caso es O(T), siendo T el número total de sistemas averiados. En espacio requiere O(1) ya que no se almacena listas intermedias en memoria.
- **Admisibilidad:** Manhattan es admisible ya que subestima el costo total por ignorar el resto de la ruta. Euclidiana es estrictamente admisible en cuadrículas con movimientos ortogonales.

Reflexión sobre el uso de inteligencia artificial

Prompt utilizado (manhattanHeuristic y euclideanHeuristic):“Revisa y optimiza esta función de heurística Manhattan/ euclidean para un problema de navegación por fases. Mantén la lógica y simplicidad del código”

La IA mencionó que es una buena heurística, y ofreció una mejora de estructura para limpiar el código.

d) systemRepairHeuristic en algorythms/heuristics.py

La heurística propuesta utiliza un Minimum Spanning Tree (MST) construido a partir de las posiciones de los sistemas averiados, el centro de control y el kit de reparación (cuando es necesario) para completar la misión. Si el robot aún no posee el kit, se suma la distancia Manhattan desde su posición hasta el kit y posteriormente se calcula el MST entre el kit, los sistemas pendientes y el centro de control. Si el kit ya fue recogido, el MST comienza desde la posición actual del robot y considera los sistemas pendientes y el centro de control.

La heurística es admisible porque utiliza distancias Manhattan como estimaciones inferiores de las distancias reales. Los obstáculos del mapa aumentan la distancia real, pero no pueden hacer que sea menor que la distancia Manhattan. Además, el MST solamente calcula el costo mínimo necesario para conectar los puntos obligatorios, sin imponer el orden de visita ni considerar todas las restricciones de la misión. Por lo tanto, representa un costo inferior del costo real:

$$h(n) \leq h'(n)$$

Comparemos las Heurísticas:

Problema: spiralWing (32 x 21 - 10 sistemas)

Heurística	Costo de Solución	Nodos Expandidos	Tiempo (seg)
systemRepairHeuristic	199	183.003	5,5
nullHeuristic	199	449.935	3,0
manhattanHeuristic	199	443.820	3,0
euclideanHeuristic	199	445.681	3,2

Problema: mazeSubsystems (39 x 23 - 12 sistemas)

Heurística	Costo de Solución	Nodos Expandidos	Tiempo (seg)
systemRepairHeuristic	225	538.920	24,8
nullHeuristic	225	2.252.527	20,6
manhattanHeuristic	225	2.231.174	21,3
euclideanHeuristic	225	2.237.373	23,0

Problema: spreadStation (41 x 23 - 13 sistemas)

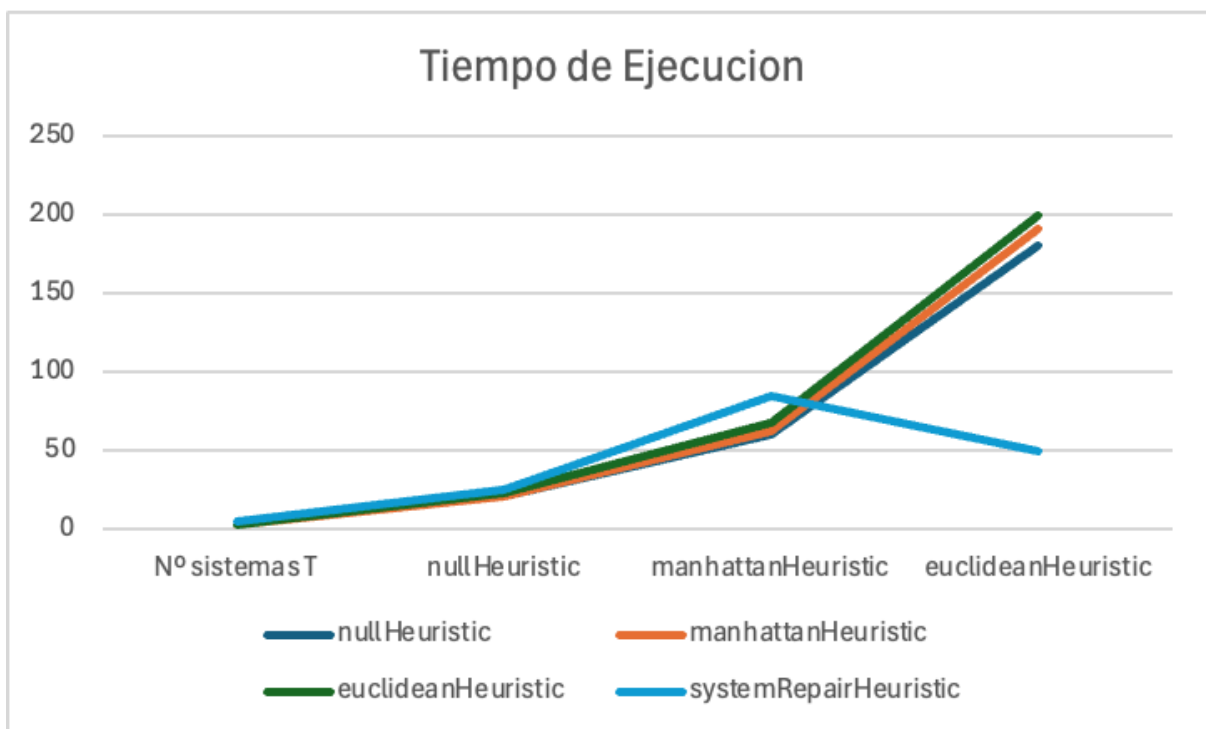
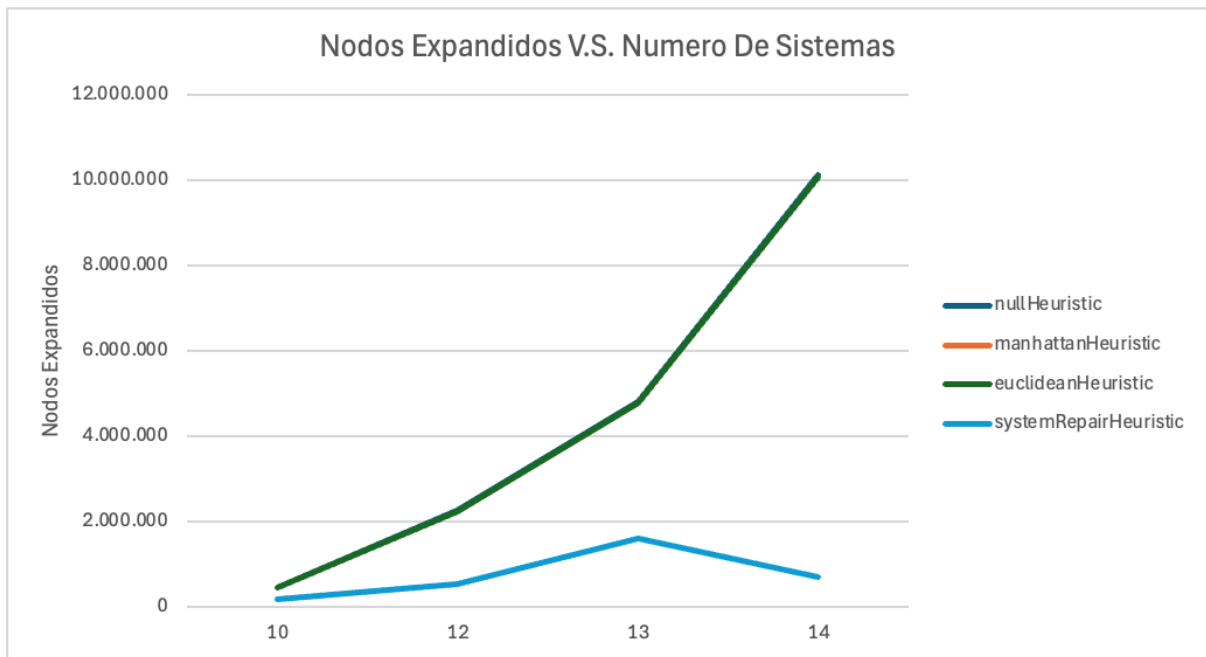
Heurística	Costo de Solución	Nodos Expandidos	Tiempo (seg)
systemRepairHeuristic	248	1.609.107	84,3
nullHeuristic	248	4.804.432	60,0
manhattanHeuristic	248	4.780.329	62,9
euclideanHeuristic	248	4.787.755	67,7

Problema: longServiceWing (43 x 23 - 14 sistemas)

Heurística	Costo de Solución	Nodos Expandidos	Tiempo (seg)
systemRepairHeuristic	208	699.208	50,0
nullHeuristic	208	10.125.238	181,0
manhattanHeuristic	208	10.061.542	191,5
euclideanHeuristic	208	10.083.341	200,2

Análisis:

Viendo los resultados podemos ver que la heurística de nosotros expande mucho menos nodos, pero como cada vez que la usamos tenemos que armar listas y usar ciclos anidados, la heurística es muy lenta para mapas que generan árboles relativamente pequeños. Sin embargo, hay un punto (entre el 3 y 4 problema) en el que está optimización genera una diferencia monumental. Esto lo podemos evidenciar en el tiempo de ejecución del 4 problema donde las demás heurísticas se tardaron entre 3 y 4 veces más. Adicionalmente, se intentó hacer los problemas de 15 sistemas sin embargo nuestra heurística fue la única que resolvió el problema en menos de 15 minutos. Nuestra heurística se tardó 561 segundos (9,35 minutos), para las demás se esperó hasta 15 minutos para ver si terminaban pero ninguna término dentro de ese periodo por lo que no usamos estos datos en el análisis.



Reflexión sobre IA:

En general la construcción de prompts fue sencilla, aunque es posible que fuera porque necesitaba cosas sencillas como la corrección de código y la organización de datos. Las correcciones que daba solían ser de errores de programación secundario como poner

“for i in len(lista)-1:”

en vez de

“for i in len(lista):”.

Las explicaciones fueron buenas, la IA explicaba porque cada error causaba problemas y escribió ejemplos y explicó cómo debería ser el flujo general del código cada vez que sugería cambios o correcciones.

Prompts:

- “Hola chat, estoy intentando escribir el código de una heurística usando MST (Minimum Spanning Tree). Necesito que me ayudes a encontrar los errores en este programa: (código)”
- “Vuelve a revisar mi código. Corregí lo que me dijiste antes. Por favor responde con pseudocódigo, quiero entender el algoritmo. Esto es lo que llevo: (código)”
- “Ahora escribe una función que use MST pero hazla lo más optimizada posible”
- "Perfecto, ahora organiza esta información del terminal en tablas, haz una tabla por problema."
- "Ahora dame una tabla para excel para que pueda mostrar el crecimiento de cada una de las medidas”
- "Organizadas por heurísticas en vez de por el mapa"
- “Genera una lista de los prompts que te he dado y una nota explicando el uso de ia que hubo en este chat”